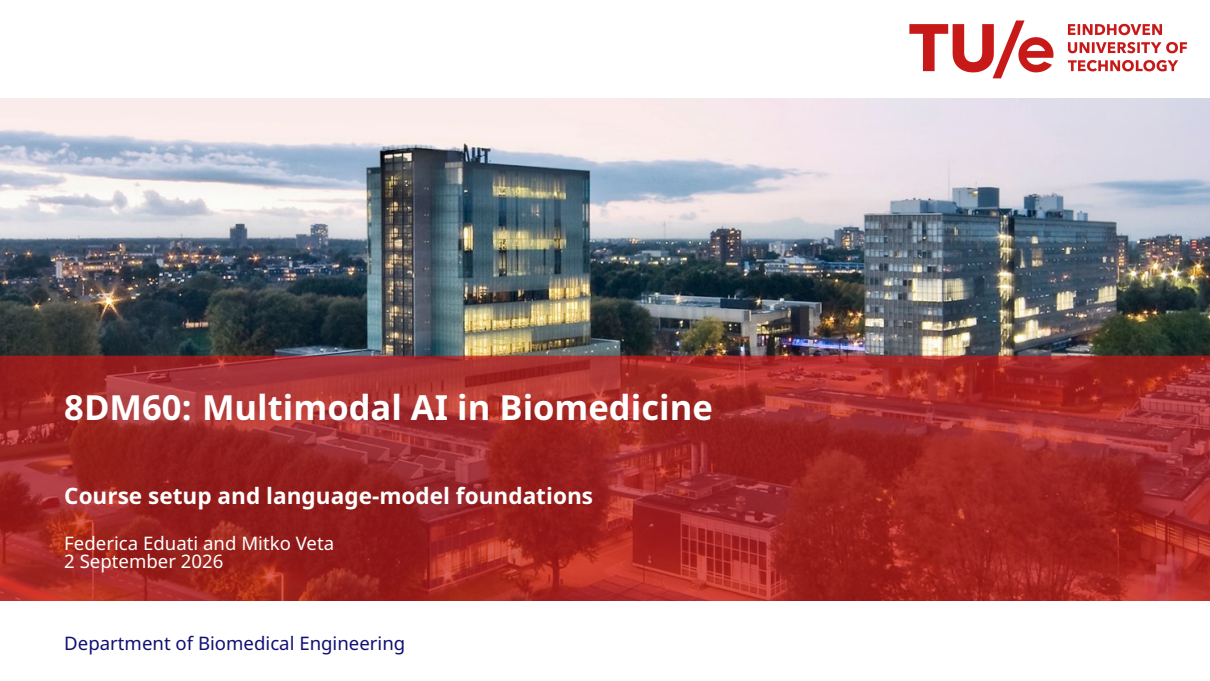


An aerial night photograph of the TU/e campus in Eindhoven, featuring several modern glass-fronted buildings with interior lights glowing. The image is partially covered by a semi-transparent red rectangle that serves as a background for the text.

# 8DM60: Multimodal AI in Biomedicine

**Course setup and language-model foundations**

Federica Eduati and Mitko Veta  
2 September 2026

# Course outline

# Lectures and guided self-study schedule

| Week | Lecture                                       | Guided self-study                      |
|------|-----------------------------------------------|----------------------------------------|
| 1    | Introduction and language models              | Introduction to project work           |
| 2    | Vision models                                 | Language models                        |
| 3    | Omics models                                  | Vision models                          |
| 4    | Models for hospital data                      | Omics models                           |
| 5    | Integration of AI in clinical workflow        | Models for hospital data               |
| 6    | Agentic AI                                    | Integration of AI in clinical workflow |
| 7    | <i>Developing AI products for healthcare*</i> | Agentic AI                             |

\*Guest-lecture content is not assessed in the written examination.

# Lectures and guided self-study have different goals

**Lecture 1** A classical lecture that introduces the course and establishes the foundations of language models and Transformers.

**Lectures 2–6** Flipped-classroom sessions. Prepare the assigned material before class; use class time to clarify concepts, compare methods, examine trade-offs, and evaluate evidence.

**Lecture 7** A guest lecture on developing AI products for healthcare.

# Guided self-study is practice for the written examination

## Consolidate knowledge

- Recall and apply methods discussed in lectures.
- Explain methodological choices and trade-offs.

## Transfer knowledge

- Analyse an unfamiliar biomedical AI system.
- Design a system, or part of one, for a biomedical use case.

## Practice for the examination

These are the two main styles of general examination question. Part of each guided self-study session is also reserved for the course project.

# How to learn in this course

## Before class

From Lecture 2, read the assigned material and note important claims, assumptions, and uncertainties.

## In class

Bring notes and questions. Actively engage in the discussion with peers and the instructor.

## In project work

Groups of five use the starter notebook to define a focused medical imaging research question and submit a one-page poster plus reproducible code.

## Assessment combines project work and written examination

| Component                             | Weight |
|---------------------------------------|--------|
| Project deliverables: poster and code | 15%    |
| Group-specific project exam questions | 15%    |
| Multiple-choice exam questions        | 20%    |
| General exam questions                | 50%    |

- **Written examination:** 28 October 2026, 13:30–16:30.
- **Resit examination:** 20 January 2027, 18:00–21:00.

# From machine learning to language models

# Learning objectives

By the end of this lecture, you should be able to:

- Explain how language modelling can be viewed as a machine-learning problem and why it requires specialised neural-network architectures.
- Describe how the Transformer architecture enables efficient and effective language modelling through attention and autoregressive prediction.
- Explain how large language models are trained from raw text using self-supervised learning.
- Describe how text is represented inside a Transformer, from tokens to contextual representations.
- Critically discuss both the strengths and limitations of next-token prediction as a learning objective.

# What do machine-learning problems have in common?

$$y = f(x)$$

| Input $x$               | Output $y$       | Task           |
|-------------------------|------------------|----------------|
| Patient characteristics | Disease category | Classification |
| Histopathology image    | Tumour subtype   | Classification |
| MRI image               | Organ mask       | Segmentation   |
| CT image                | Survival risk    | Regression     |

In supervised learning, examples  $\{(x_i, y_i)\}_{i=1}^n$  are used to learn a function  $f$ .

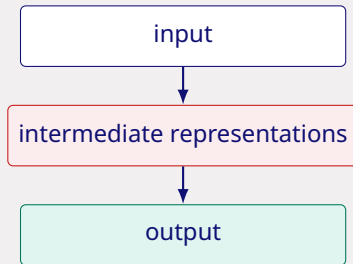
**Next question:** What form can the learned function  $f$  take?

# What form can the learned function take?

## Possible model families

- Linear regression
- Logistic regression
- Decision trees
- Random forests
- Neural networks

## Neural-network example

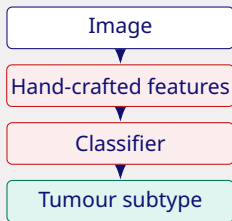


**A neural network jointly learns intermediate representations and a task-specific mapping.**

**Next question:** What useful information can those intermediate representations encode?

# What does a neural network learn between input and output?

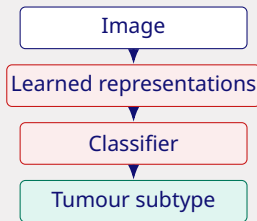
## Traditional machine learning



Features are designed before classifier training.

VS.

## Deep learning



Representations and the classifier are learned jointly.

**Deep networks learn task-useful representations directly from data.**

**Next question:** Can we use exactly the same neural network for every type of data?

# Can we use exactly the same neural network for every type of data?

Patient table

Histopathology image

Molecular graph

Clinical text

## Which architecture would you choose for each input?

Think for 30 seconds before the architectures are revealed.

**Next question:** What structure does language have, and what machine-learning task should a language model solve?

# Can we use exactly the same neural network for every type of data?

| Data                 | Relevant structure                     | Architecture                |
|----------------------|----------------------------------------|-----------------------------|
| Patient table        | Fixed-length features                  | Multilayer perceptron (MLP) |
| Histopathology image | Spatial locality and repeated patterns | Convolutional network (CNN) |
| Molecular graph      | Relations defined by edges             | Graph neural network (GNN)  |
| Clinical text        | Order and contextual dependencies      | Transformer                 |

**Inductive biases build useful assumptions about the data into the architecture.**

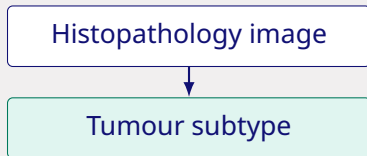
**Next question:** What structure does language have, and what machine-learning task should a language model solve?

# **What is the machine-learning task for language?**

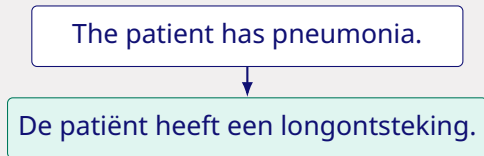
Language is sequential—but what should the model predict?

# Why is translation different from classification?

**Classification predicts a label**



**Translation predicts a sequence**



**Both the input and output are ordered sequences, and they can have different lengths.**

**Next question:** How should we design a neural network that understands one sentence and generates another?

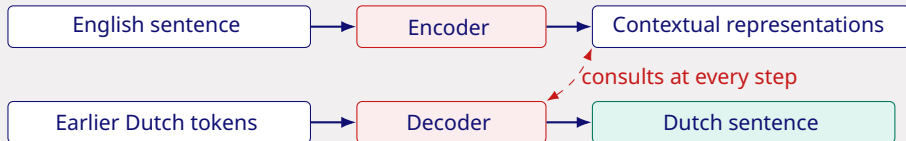
## Which language tasks map one sequence to another?

| Task                | Input sequence                    | Output sequence              |
|---------------------|-----------------------------------|------------------------------|
| Translation         | Sentence in one language          | Sentence in another language |
| Summarization       | Long document                     | Short summary                |
| Grammar correction  | Uncorrected sentence              | Corrected sentence           |
| Report generation   | Clinical findings or observations | Written report               |
| Text simplification | Complex text                      | Simplified text              |

**The shared challenge is to represent the input sequence and generate a new output sequence.**

**Next question:** What architecture can perform both operations?

# How can a model understand one sentence and generate another?



## Encoder

- Reads the complete English sentence.
- Produces one contextual representation per input position.

## Decoder

- Uses the source representations and earlier Dutch tokens.
- Predicts the next output token.

**This encoder-decoder setting motivated the original Transformer for machine translation.**

**Next question:** How can the decoder generate an entire sentence?

# How can the decoder generate an entire sentence?

The translation is constructed incrementally:

De

$$P(y_t \mid y_{<t}, x)$$

- $x$ : source sentence
- $y_{<t}$ : previous output tokens
- $y_t$ : next output token

**At every step, the decoder conditions its prediction on the English source and the Dutch output generated so far.**

**Next question:** What happens if we remove the source sentence?

# How can the decoder generate an entire sentence?

The translation is constructed incrementally:

De

De patiënt

$$P(y_t \mid y_{<t}, x)$$

- $x$ : source sentence
- $y_{<t}$ : previous output tokens
- $y_t$ : next output token

**At every step, the decoder conditions its prediction on the English source and the Dutch output generated so far.**

**Next question:** What happens if we remove the source sentence?

# How can the decoder generate an entire sentence?

The translation is constructed incrementally:

- De
- De patiënt
- De patiënt heeft
- $x$ : source sentence
  - $y_{<t}$ : previous output tokens
  - $y_t$ : next output token

$$P(y_t \mid y_{<t}, x)$$

**At every step, the decoder conditions its prediction on the English source and the Dutch output generated so far.**

**Next question:** What happens if we remove the source sentence?

# How can the decoder generate an entire sentence?

The translation is constructed incrementally:

$$P(y_t \mid y_{<t}, x)$$

- De  
De patiënt  
De patiënt heeft  
De patiënt heeft een
- $x$ : source sentence
  - $y_{<t}$ : previous output tokens
  - $y_t$ : next output token

**At every step, the decoder conditions its prediction on the English source and the Dutch output generated so far.**

**Next question:** What happens if we remove the source sentence?

# How can the decoder generate an entire sentence?

The translation is constructed incrementally:

$$P(y_t \mid y_{<t}, x)$$

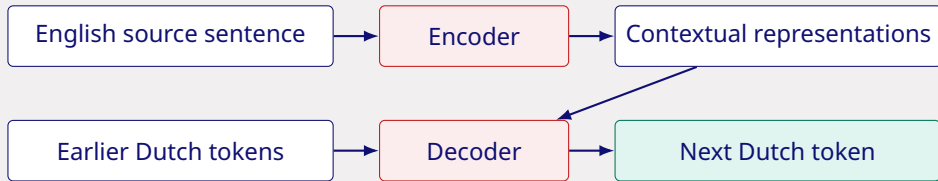
- De
- De patiënt
- De patiënt heeft
- De patiënt heeft een
- De patiënt heeft een longontsteking
- $x$ : source sentence
  - $y_{<t}$ : previous output tokens
  - $y_t$ : next output token

**At every step, the decoder conditions its prediction on the English source and the Dutch output generated so far.**

**Next question:** What happens if we remove the source sentence?

# What happens if we remove the source sentence?

Translation uses an encoder and a decoder

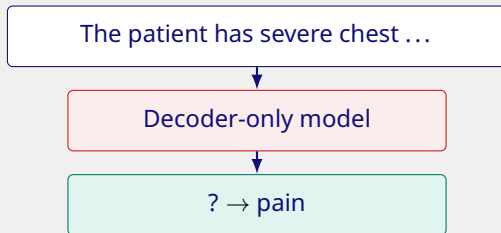


The source sentence supplies context through the encoder.

**Next question:** How can one sequence provide many examples for this learning objective?

# What happens if we remove the source sentence?

**Encoder and source sentence removed**



$$P(x_t | x_{<t})$$

**The decoder now predicts the continuation of its own input. This is the basic GPT-style language-model architecture.**

**Next question:** How can one sequence provide many examples for this learning objective?

## How can one sequence supply many training examples?

| Input                        | Target  |
|------------------------------|---------|
| The                          | patient |
| The patient                  | has     |
| The patient has              | severe  |
| The patient has severe       | chest   |
| The patient has severe chest | pain    |

$$P(x_1, \dots, x_T) = \prod_{t=1}^T P(x_t \mid x_{<t})$$

**The probability of a sequence is decomposed into a series of next-token predictions.**

**Next question:** Where do the targets come from when nobody labels the text?

# Where do the targets come from when nobody labels the text?

## Observed text

The biopsy showed invasive carcinoma

## Derived training example

**Input:** The biopsy showed invasive  
**Target:** carcinoma

## The optimization remains supervised.

- Conventional supervised learning uses externally provided targets.
- Self-supervised learning derives targets automatically from observed data.

This makes learning from very large text collections possible without manual labels for every example.

**Next question:** What makes this a generative rather than a fixed-output task?

# What makes language modelling a generative task?

## Discriminative modelling

$$P(y | x)$$

**Examples:** classification, regression, and segmentation.

Image  $\rightarrow$  tumour subtype

**Nuance:** conditional generation can also use  $P(y | x)$ , but it produces a structured sequence rather than a fixed class or scalar.

## Generative modelling

$$P(x_1, \dots, x_T)$$

**Examples:** language modelling, translation, and report generation.

Sequence beginning  $\rightarrow$  continuation

**Next question:** How can a computer represent the words in these sequences?

## How can a computer represent words?



1. A vocabulary defines the available tokens.
2. The tokenizer divides text into those tokens.
3. Each token is mapped to an arbitrary integer identifier.
4. Each identifier selects a learned numerical vector.

**Next question:** How can a tokenizer handle a long medical word?

# How can a tokenizer handle a long medical word?

**Text:** immunohistochemistry

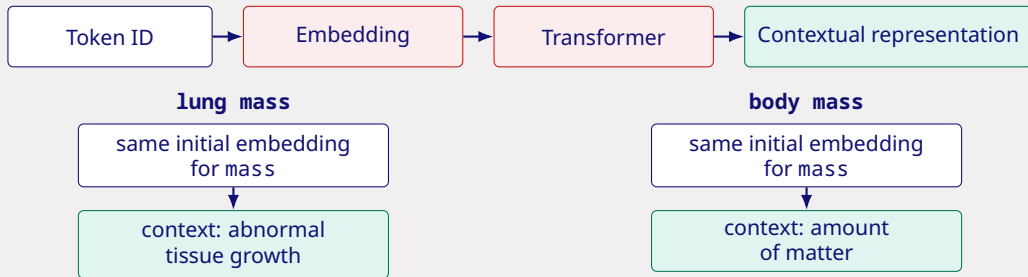
**One possible split:**



- Tokenizers can represent unfamiliar words by combining reusable word pieces.
- Different tokenizers may split immunohistochemistry differently.
- The resulting token IDs are arbitrary indices, not measurements of meaning.

**Next question:** Does a token embedding already contain the complete meaning of a token?

# Does a token embedding already contain its complete meaning?



**The embedding is fixed for the token; the Transformer produces a context-dependent representation.**

**Next question:** Which sequence architecture should process these token representations?

# Why not process language with a CNN or an RNN?

**CNNs emphasize local patterns; language also requires direct access to distant context.**

## **Recurrent neural networks**

- Are designed for sequences.
- Support variable-length input.
- Require sequential computation.
- Are difficult to parallelize.
- Can struggle with long-range dependencies.

## **Transformers**

- Support parallel training across sequence positions.
- Allow direct interaction between distant tokens.
- Scale more effectively with data and compute.

**Next question:** What capabilities must a Transformer provide for language modelling?

# What must our language model be able to do?

## Propose the engineering requirements

- Represent token order.
- Use context and connect distant words.
- Focus on relevant words and ignore irrelevant ones.
- Avoid future information during next-token prediction.
- Process long sequences efficiently.
- Support stable training of deep networks.

Discuss with a neighbour: which mechanism might satisfy each requirement?

**Next question:** Which attention operations are required in the original encoder-decoder Transformer?

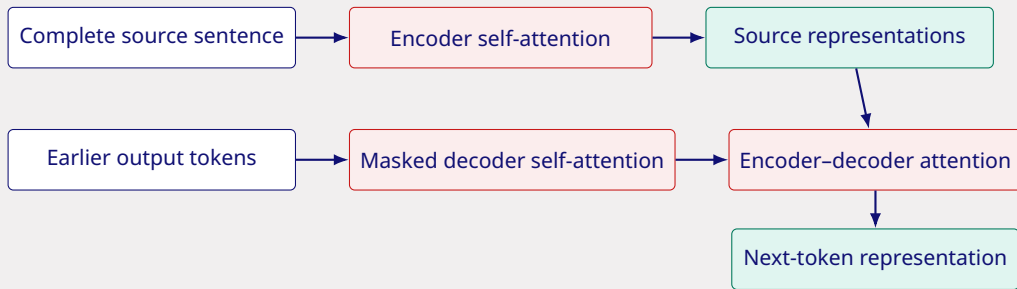
# What must our language model be able to do?

| Requirement                                | Transformer mechanism                  |
|--------------------------------------------|----------------------------------------|
| Represent order                            | Positional information                 |
| Connect tokens and select relevant context | Self-attention                         |
| Ignore future tokens                       | Causal mask                            |
| Transform representations                  | Feed-forward network                   |
| Support stable deep training               | Residual connections and normalization |

**The Transformer is a response to these engineering requirements, not a collection of arbitrary components.**

**Next question:** Which attention operations are required in the original encoder-decoder Transformer?

## Which attention operations does translation require?

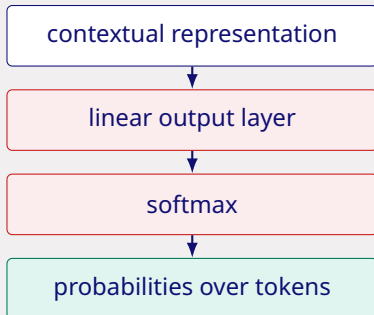


- Encoder self-attention uses the complete source sentence.
- Masked decoder self-attention uses only earlier output tokens.
- Encoder-decoder attention selects relevant source information at every generation step.

**Decoder-only LLMs mainly retain masked self-attention over the context supplied to the model.**

**Next question:** How does the final contextual representation become a next-token prediction?

## The final contextual representation becomes a token distribution



|              |      |
|--------------|------|
| pain         | 0.61 |
| pressure     | 0.17 |
| discomfort   | 0.11 |
| pneumonia    | 0.04 |
| other tokens | 0.07 |

Training increases the probability assigned to the observed next token.

**Next question:** What prevents the model from seeing that token during training?

# What prevents the model from cheating during training?

The complete sequence is available for efficient training, but a causal mask blocks every future position.

| Representation at | The | patient | has | chest | pain |
|-------------------|-----|---------|-----|-------|------|
| The               | ✓   | ×       | ×   | ×     | ×    |
| patient           | ✓   | ✓       | ×   | ×     | ×    |
| has               | ✓   | ✓       | ✓   | ×     | ×    |
| chest             | ✓   | ✓       | ✓   | ✓     | ×    |
| pain              | ✓   | ✓       | ✓   | ✓     | ✓    |

✓: visible context

×: hidden future token

**Next question:** If training predicts all positions together, why must generation still proceed one token at a time?

# Training predicts in parallel; generation remains sequential

## During training

1. Provide the complete sequence.
2. Predict the next token at every position.
3. Use a causal mask to hide future tokens.
4. Compute loss over all positions.

many next-token predictions  
in parallel

## During generation

1. Predict one token.
2. Append it to the sequence.
3. Process the extended context.
4. Repeat.

one new token  
at a time

**Next question:** Why does this apparently simple objective teach the model so much?

# Why does predicting the next token teach the model so much?

Accurate prediction can require patterns involving:

- grammar and semantics;
- factual associations;
- medical terminology;
- document structure;
- distant dependencies.

The same learned patterns support:

- translation and summarization;
- question answering;
- report generation;
- information extraction;
- dialogue.

**A simple objective can demand rich internal representations when applied at scale.**

**Next question:** What does fluent next-token prediction still fail to guarantee?

# What does fluent next-token prediction fail to guarantee?

The objective does not guarantee:

- factual correctness;
- causal understanding;
- reliable reasoning;
- calibrated uncertainty.

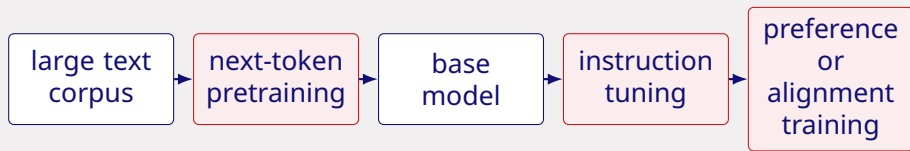
Nor does it guarantee:

- faithful explanations;
- clinical safety;
- alignment with user intentions.

**In medicine, linguistic plausibility is not evidence of clinical correctness.**

**Next question:** How can a base language model be adapted into a useful assistant?

# How does a base language model become an assistant?



## Optional domain adaptation

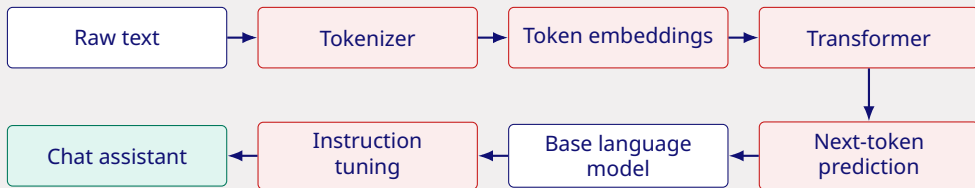
Further training can specialize the model using biomedical literature or clinical text.

## Retrieval-augmented generation

Retrieval supplies external information in the context at inference time; it does not necessarily change the model weights.

**Next question:** How do tokenization, pretraining, and adaptation fit into one complete pipeline?

## How do the pieces fit into one complete pipeline?



**Raw text supplies both the input and the next-token targets; adaptation then turns the pretrained base model into an interactive assistant.**

**Next question:** Where do these foundations lead in medical AI?

# Where do these foundations lead in medical AI?

- Medical language models
- Clinical question answering
- Report generation
- Retrieval-augmented generation
- Vision-language models
- Multimodal foundation models
- Instruction tuning for medical tasks
- Evaluation and safety

**Throughline:** machine learning learns functions; neural networks learn representations; Transformers learn contextual sequence representations through next-token prediction.

**Next question:** How do we transform a general foundation model into a useful biomedical model?

# **Adapting foundation models for biomedical applications**

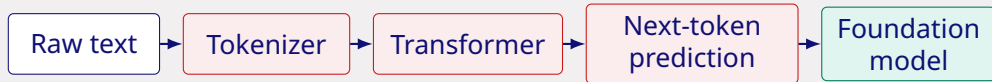
From a general pretrained model to a task-specific biomedical system

# Learning objectives

By the end of this section, you should be able to:

- Compare prompting, retrieval-augmented generation, and fine-tuning as adaptation strategies.
- Explain why full fine-tuning is expensive and how parameter-efficient fine-tuning reduces trainable parameters.
- Derive the LoRA parameterization and explain its rank, scaling, target modules, and inference options.
- Explain quantization and how QLoRA combines a quantized frozen base model with trainable LoRA adapters.
- Distinguish parameter adaptation from the training objective, including instruction tuning.

## How does a foundation model become a biomedical model?



**Self-supervised learning creates a capable general model, not a model developed for one biomedical application.**

**Next question:** How do we adapt this model to a biomedical application?

# Why not train a new foundation model?

## Typical training scale

- Trillions of tokens
- Billions of parameters
- Thousands of GPUs
- Weeks to months of training

## Typical pretraining corpora

- Web pages and books
- Scientific papers
- Code
- Wikipedia

**Training a new foundation model is beyond the resources of almost every research group or hospital.**

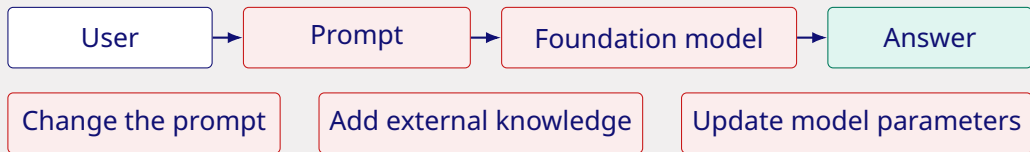
**Next question:** If we cannot train a new model, what can we change instead?

## What parts of the system can we modify?



**Next question:** Can changing only the input improve biomedical performance?

## What parts of the system can we modify?



**These choices lead to prompting, retrieval-augmented generation, and fine-tuning.**

**Next question:** Can changing only the input improve biomedical performance?

# Can changing only the prompt improve performance?

The same clinical case can be paired with different instructions:

---

- 1 Summarise the report.
  - 2 Act as a pulmonologist and summarise the report.
  - 3 Generate differential diagnoses and explain the evidence for each.
- 

## Advantages

- No retraining
- Immediate
- Very low computational cost

## Limitations

- Cannot reliably add knowledge the model never learned
- Cannot permanently change model behaviour
- Can be sensitive to wording

**Next question:** What if the model has never seen the required biomedical knowledge?

# How can retrieval add biomedical knowledge at inference time?



**Biomedical sources:** PubMed articles, clinical guidelines, electronic health records, hospital protocols, and drug databases.

## Advantages

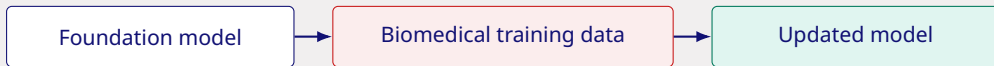
- Supports current and patient-specific knowledge
- Enables source attribution
- Does not update model weights

## Limitations

- Retrieval repeats for every query
- Poor retrieval produces poor answers
- Documents may conflict

**Next question:** What if we want the model itself to change?

# When should we update the model itself?



## Biomedical examples

- Radiology or pathology report generation
- Clinical coding
- Biomedical question answering
- Structured clinical-text extraction

## Fine-tuning may change

- Behaviour and style
- Task performance
- Domain-specific representations

**Fine-tuning does not automatically guarantee accurate or current factual knowledge.**

**Next question:** Why does not everyone fully fine-tune large language models?

# Why is full fine-tuning expensive?

## Training must store

- Model parameters
- Gradients
- Optimizer states
- Intermediate activations

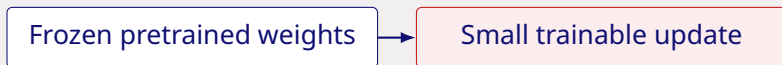
## A 7-billion-parameter example

- FP16 weights alone: roughly 14 GB
- Gradients require additional memory
- Adam states can require several more parameter-sized copies
- Activations grow with batch size and sequence length

**Training requires substantially more memory than inference.**

**Next question:** Do we really need to update every parameter?

# Can we adapt a model without updating every parameter?



## Parameter-efficient fine-tuning

- Freezes most or all pretrained weights.
- Trains only a small number of additional parameters.

## PEFT methods

- Adapters
- Prefix tuning
- Prompt tuning
- LoRA

**PEFT is a family of methods; LoRA is one particularly popular member.**

**Next question:** How can a useful update use far fewer trainable parameters?

# How does LoRA separate pretrained knowledge from adaptation?

One Transformer linear layer

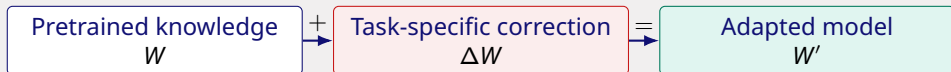
$$y = Wx, \quad W \in \mathbb{R}^{d \times k}$$

LoRA freezes  $W$  and learns an update

$$W' = W + \Delta W$$

Full fine-tuning may update every element of  $W$ .

$$y = (W + \Delta W)x$$



**Next question:** Can the update matrix  $\Delta W$  be represented more compactly?

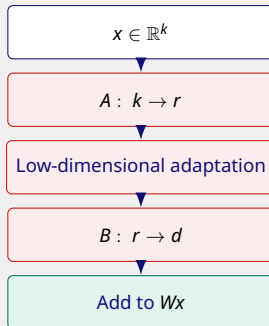
# How does the low-rank assumption compress the update?

$$\Delta W = BA$$

$$B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times k}$$

$$r \ll \min(d, k)$$

$$y = Wx + BAx$$



LoRA replaces one large task-specific update with two projections through a much smaller dimension  $r$ .

**Next question:** How is the low-rank update scaled and initialized?

# How is the LoRA update scaled and initialized?

## Scaled update

$$\Delta W = \frac{\alpha}{r} BA$$

$$y = Wx + \frac{\alpha}{r} BAx$$

- $r$ : LoRA rank
- $\alpha$ : adaptation magnitude
- $\alpha/r$ : stabilizes scale across ranks

## Training and initialization

- $W$  remains frozen.
- Only  $A$  and  $B$  receive gradients.
- Commonly, one matrix starts random and the other starts at zero.
- Therefore  $\Delta W = 0$  initially, so behaviour starts from the pretrained model.

**Next question:** How many parameters does LoRA actually train?

# Why does LoRA train so few parameters?

## Full fine-tuning

$$W \in \mathbb{R}^{d \times k}$$

$$\text{trainable parameters} = dk$$

**Example:**  $d = k = 4096$ ,  $r = 16$

$$4096^2 = 16,777,216$$

$$16(4096 + 4096) = 131,072$$

## LoRA

$$\text{trainable parameters} = dr + rk = r(d + k)$$

$$\frac{131,072}{16,777,216} \approx 0.0078$$

**LoRA trains less than 1% as many parameters for this matrix. It does not make the base model small; it makes the trainable update small.**

**Next question:** Which Transformer matrices should receive these updates?

# Where can LoRA be applied inside a Transformer?

## Common targets

- Query projection  $W_Q$
- Key projection  $W_K$
- Value projection  $W_V$
- Attention output projection  $W_O$
- Feed-forward projections

## Attention reminder

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

## LoRA on the query projection

$$Q = X(W_Q + B_Q A_Q)$$

**Adapting more layers or matrix types increases capacity, trainable parameters, memory use, and training time.**

**Next question:** Which LoRA hyperparameters control this trade-off?

# Which choices control LoRA behaviour?

## Rank $r$

- Small: fewer parameters, less capacity
- Large: more parameters, more capacity

## Scaling $\alpha$

Controls update magnitude.

## LoRA dropout

Regularizes the LoRA branch during training.

## Target modules

Select which weight matrices receive adapters.

## Performance also depends on

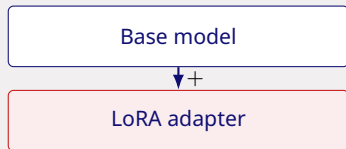
- Adapted layers and matrices
- Learning rate
- Training-data quality
- Training objective

**Next question:** What happens to the LoRA adapters at inference time?

# How can LoRA be used at inference time?

$$y = Wx + BAx$$

## Keep adapters separate



- One base model supports multiple tasks.
- Small adapters can be switched dynamically.

**LoRA is efficient because the adaptation is small, not because the base model disappears.**

**Next question:** If the base model remains, how can its memory footprint be reduced?

## Merge the update

$$W' = W + BA, \quad y = W'x$$

- No separate LoRA branch.
- No additional adapter matrix operations.

## Why is the frozen base model still large?

$$\text{weight memory} = N \times \text{bits per parameter}$$

| Representation | 7B-model weight memory |
|----------------|------------------------|
| FP32           | 28 GB                  |
| FP16/BF16      | 14 GB                  |
| INT8           | 7 GB                   |
| INT4           | 3.5 GB                 |

**These values describe weights only; runtime memory can be larger.**

**Next question:** Can we represent model weights using fewer bits?

## How does quantization represent weights using fewer bits?

Quantization maps a high-precision weight  $w$  to a discrete integer value  $q$ :

$$q = \text{round} \left( \frac{w}{s} \right) + z$$

where  $s$  is a scale and  $z$  is a zero point.

The approximate reconstructed weight is:

$$\hat{w} = s(q - z)$$



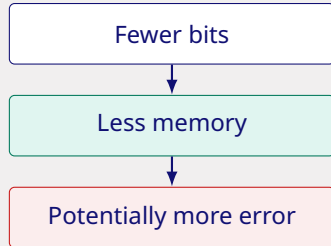
**Next question:** What accuracy do we lose when using fewer levels?

# What trade-off does quantization introduce?

$$e = w - \hat{w}$$

## Using fewer bits gives

- Fewer representable values
- Larger potential approximation error
- Lower memory use



**Quality depends on bit width, quantization scheme, calibration data, quantized tensors, and the weight distribution.**

**Next question:** How should the discrete levels be positioned around zero?

# When should quantization use a zero point?

## Symmetric quantization

$$q = \text{round} \left( \frac{W}{S} \right)$$

- Levels are represented around zero.
- The formulation is simple and often efficient.

## Asymmetric quantization

$$q = \text{round} \left( \frac{W}{S} \right) + z$$

- Uses a nonzero zero point  $z$ .
- Can better represent distributions not centred around zero.

**The central principle is that a continuous range is approximated by a finite set of discrete levels.**

**Next question:** Should every weight share the same scale?

# At what granularity should the quantization scale be computed?

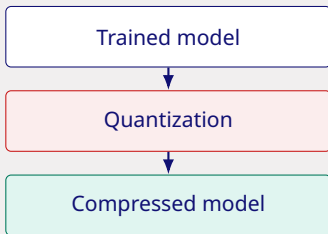
| Granularity | Scale assignment                  | Trade-off                                        |
|-------------|-----------------------------------|--------------------------------------------------|
| Per tensor  | One scale for the complete tensor | Simple and fast, but potentially less accurate   |
| Per channel | One scale for each output channel | Adapts to different ranges; stores more scales   |
| Group-wise  | One scale for each weight group   | Common in LLMs; balances accuracy and efficiency |

**Quantization is not only a choice between 4-bit and 8-bit storage; the grouping strategy also matters.**

**Next question:** Should quantization happen after training or during training?

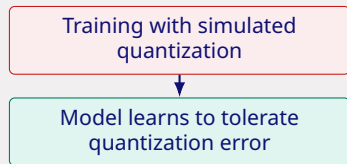
# When should quantization be introduced?

## Post-training quantization



- No retraining or limited calibration
- Simple deployment pipeline

## Quantization-aware training



- Can preserve quality better
- Requires additional training

**Next question:** When a model is called “4-bit,” what exactly is quantized?

# What exactly can be quantized?

## Possible targets

- Model weights
- Activations
- Key-value cache
- Gradients
- Optimizer states

## A common LLM deployment

- Stores weights in 4-bit form.
- Dequantizes weights for computation.
- Uses FP16 or BF16 accumulation for matrix multiplication.

**A “4-bit model” does not necessarily perform every operation using 4-bit arithmetic.**

**Next question:** How can quantization and LoRA solve complementary bottlenecks?

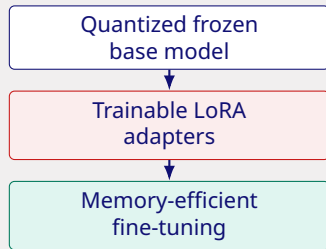
# Why combine LoRA and quantization?

## LoRA reduces

- The number of trainable parameters
- Gradient and optimizer-state memory

## Quantization reduces

- Memory for the frozen base model
- The hardware needed to hold the model



**The methods address complementary bottlenecks: frozen-model storage and adaptation cost.**

**Next question:** How does QLoRA keep the base model quantized while training adapters?

## How does QLoRA work?

$$\hat{W} = D(Q(W))$$

$$y = \hat{W}x + BAx$$

- $Q$  stores the base weights in low precision.
- $D$  dequantizes them for computation.

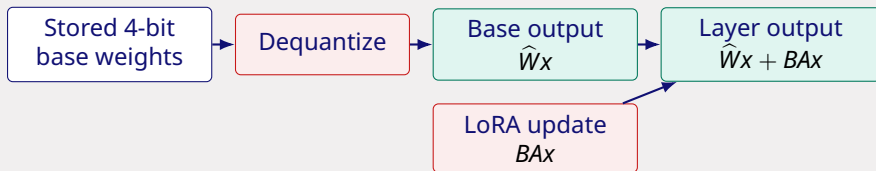
### During training

- The quantized base model remains frozen.
- LoRA adapters use FP16 or BF16.
- Gradients flow only to the adapters.
- The base weights are not updated.

**QLoRA = a quantized frozen base model + trainable LoRA adapters + higher-precision computation.**

**Next question:** What is the exact data flow through a QLoRA layer?

# What happens inside a QLoRA layer?



## Stored compactly

- Frozen base-model weights
- Quantization scales and metadata

**QLoRA does not directly train the 4-bit base weights.**

## Computed with more precision

- Dequantized matrix products
- Trainable adapter updates

**Next question:** What problem does each adaptation method solve?

# Which method solves which bottleneck?

| Method           | What changes                      | Main problem addressed                                               |
|------------------|-----------------------------------|----------------------------------------------------------------------|
| Full fine-tuning | All model parameters              | Maximum adaptation flexibility when compute and memory are available |
| PEFT             | A small parameter subset          | Avoid updating and storing optimizer state for the complete model    |
| LoRA             | Low-rank weight updates           | Efficient parameter adaptation with small, reusable adapters         |
| Quantization     | Numerical representation          | Reduce model memory and often deployment cost                        |
| QLoRA            | LoRA over a quantized frozen base | Reduce both adaptation cost and frozen-model memory                  |

**Efficiency methods determine how we adapt; the learning objective determines what behaviour is learned.**

**Next question:** Parameters can change, but what behaviour should the model learn?

# What objective should fine-tuning optimize?

## Possible objectives

- Continued next-token prediction
- Classification
- Translation
- Summarization

## Interactive objectives

- Question answering
- Dialogue
- Instruction following

**The same adaptation mechanism can optimize different objectives; the task data and loss define the desired behaviour.**

**Next question:** How do modern assistants learn to respond to instructions?

# How does instruction tuning differ from pretraining?

## Pretraining objective

$$\mathcal{L}_{\text{pre}} = - \sum_t \log p_{\theta}(x_t \mid x_{<t})$$

- Predict the continuation of text.
- Learn broad linguistic and factual patterns.

## Instruction-tuning objective

$$\mathcal{L}_{\text{inst}} = - \sum_t \log p_{\theta}(y_t \mid x, y_{<t})$$

- Condition on an instruction  $x$ .
- Learn to produce the desired response  $y$ .

**Instruction tuning is an objective and data format, not a compression method.** It can use full fine-tuning, LoRA, QLoRA, or another PEFT method.

**Next question:** How do these choices fit into one conceptual framework?

# How do the adaptation choices fit together?

| Question                                         | Candidate approach                                                   |
|--------------------------------------------------|----------------------------------------------------------------------|
| What part of the system should change?           | Prompting, retrieval-augmented generation, or fine-tuning            |
| What behaviour should be learned?                | Choose the objective and, where appropriate, instruction-tuning data |
| How can fewer parameters be trained?             | Parameter-efficient fine-tuning, especially LoRA                     |
| How can model memory be reduced?                 | Quantization                                                         |
| How can both adaptation and memory be efficient? | QLoRA                                                                |

**Separate the modelling decision from the systems decision:**  
*what should the model learn, and how can we afford to make it learn?*

**Next question:** What sequence of decisions leads to a suitable adaptation strategy?

# How should we choose an adaptation strategy?

## Start with the smallest necessary change

1. Can prompting produce the desired behaviour?  
→ Use prompting.
2. Does the model need current or external knowledge?  
→ Add retrieval-augmented generation.
3. Must the model's behaviour change?  
→ Fine-tune for continuation, classification, generation, or instruction following.

**If no model change is needed, reconsider whether fine-tuning is justified at all.**

## Then match training to resources

1. Is full fine-tuning affordable?  
→ Update the complete model.
2. Can the update be parameter-efficient?  
→ Use PEFT; use LoRA for a low-rank update.
3. Is frozen-model memory still the bottleneck?  
→ Combine quantization and LoRA as QLoRA.

**Next question:** After choosing an efficient adaptation method, how do we establish that the resulting system is trustworthy?

# Adaptation is only the beginning

**We now understand how to adapt and compress a foundation model.**

The next question is not only whether it is trainable, but whether it is trustworthy.

## Evaluation must establish

- Evidence for performance and generalization
- Known limitations and failure modes

## Deployment must account for

- Human and tool interaction
- Monitoring after release

**Efficient adaptation makes a system feasible; rigorous evaluation determines whether it is fit for use.**